

Name : A Syed Thoufiq

Reg No : 192424211

Course/Code : DSA0604 - DATA HANDLING AND VISUALIZATION

Date : 21/08/26

QUESTION 1: Daily Sales Time Series – Retail Chain (R)

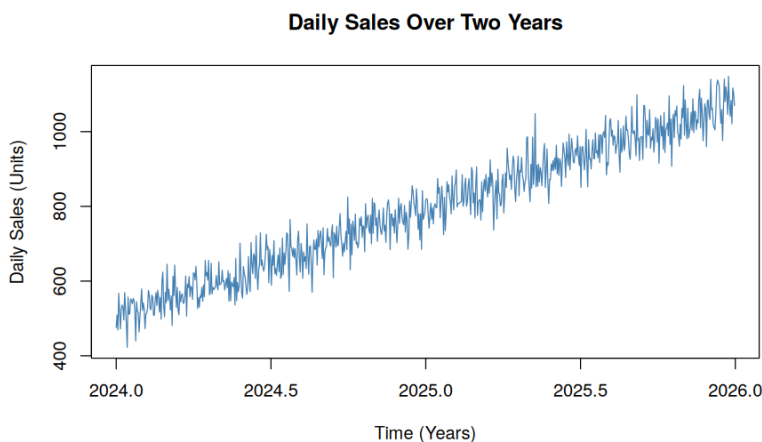
1. Code:

```
sales <- 500 + 0.8*(1:730) + rnorm(730, 0, 40)
sales_ts <- ts(sales, frequency = 365, start = c(2024,1))
```

```
plot(sales_ts,
     xlab = "Time (Years)",
     ylab = "Daily Sales (Units)",
     main = "Daily Sales Over Two Years",
     col = "steelblue")
```

```
abline(reg = lm(sales ~ c(1:730))), col = "red", lwd = 2)
```

Output:



Interpretation:

The daily sales fluctuate a lot day to day, but the red trend line shows a clear steady upward movement over the two years. This long-term growth is useful for the finance team when setting next year's budget, since it suggests demand is genuinely increasing rather than staying flat.

QUESTION 2: Hourly Server Temperature – IoT Monitoring (Python)

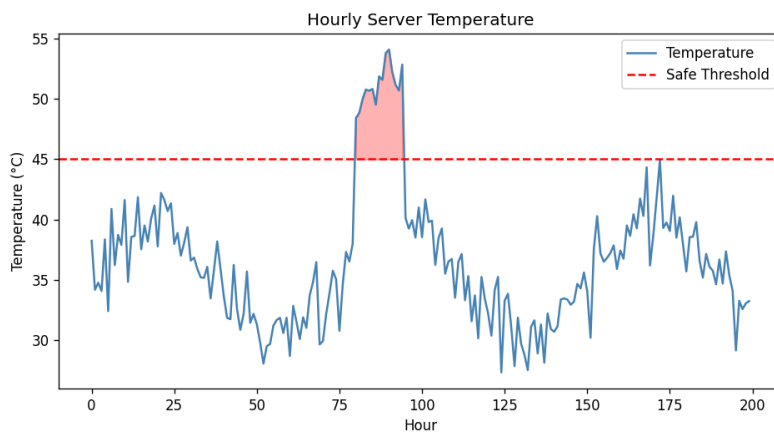
1. Code:

```
import numpy as np
import matplotlib.pyplot as plt

hours = np.arange(200)
temp = 35 + 5*np.sin(hours/12) + np.random.normal(0, 2, 200)
threshold = 45

plt.plot(hours, temp, color="steelblue", label="Temperature")
plt.axhline(threshold, color="red", linestyle="--", label="Safe Threshold")
plt.fill_between(hours, temp, threshold, where=(temp>threshold), color="red", alpha=0.3)
plt.xlabel("Hour")
plt.ylabel("Temperature (°C)")
plt.title("Hourly Server Temperature")
plt.legend()
plt.show()
```

Output:



Interpretation:

The temperature mostly stays within the safe range and cycles up and down normally. Around hour 80–95 it spikes well above the 45°C threshold, shown by the shaded red region – this is the overheating window the IoT team should be alerted about.

QUESTION 3: Closing Prices of Three Tech Stocks (R)

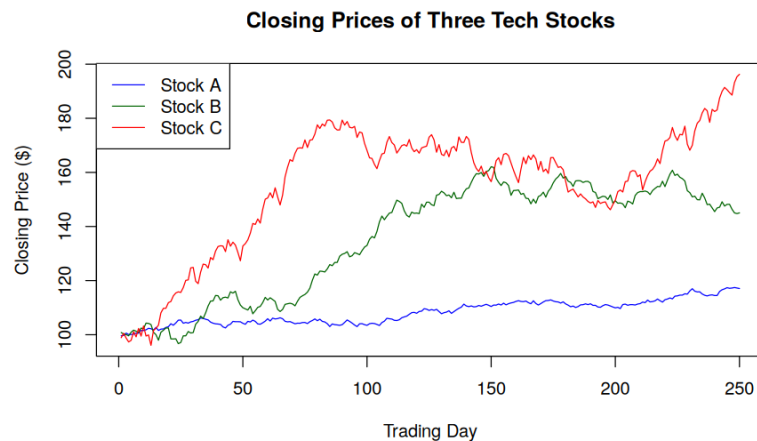
1. Code:

```
days <- 1:250
StockA <- 100 + cumsum(rnorm(250, 0.05, 0.5))
StockB <- 100 + cumsum(rnorm(250, 0.05, 1.5))
StockC <- 100 + cumsum(rnorm(250, 0.05, 2.5))

plot(days, StockA, type = "l", col = "blue",
      ylim = range(c(StockA, StockB, StockC)),
      xlab = "Trading Day", ylab = "Closing Price ($)",
      main = "Closing Prices of Three Tech Stocks")
lines(days, StockB, col = "darkgreen")
lines(days, StockC, col = "red")

legend("topleft", legend = c("Stock A", "Stock B", "Stock C"),
      col = c("blue", "darkgreen", "red"), lty = 1)
```

Output:



Interpretation:

Stock A (blue) moves in the smoothest, tightest path with the least day-to-day swing, while Stock C (red) swings the widest. This means Stock A shows the steadiest growth and is the least volatile of the three, making it the safer pick for an investor who wants stability.

QUESTION 4: Monthly Electricity Consumption – Four Neighborhoods (Python)

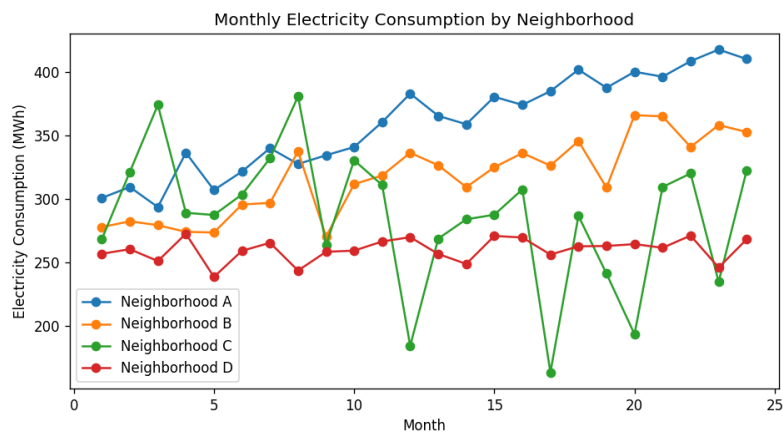
1. Code:

```
import numpy as np
import matplotlib.pyplot as plt

months = np.arange(1, 25)
A = 300 + 5*months + np.random.normal(0, 10, 24)
B = 280 + 3*months + np.random.normal(0, 15, 24)
C = 320 - 2*months + np.random.normal(0, 60, 24)
D = 260 + np.random.normal(0, 8, 24)

plt.plot(months, A, label="Neighborhood A", marker="o")
plt.plot(months, B, label="Neighborhood B", marker="o")
plt.plot(months, C, label="Neighborhood C", marker="o")
plt.plot(months, D, label="Neighborhood D", marker="o")
plt.xlabel("Month")
plt.ylabel("Electricity Consumption (MWh)")
plt.title("Monthly Electricity Consumption by Neighborhood")
plt.legend()
plt.show()
```

Output:



Interpretation:

Neighborhood D stays flat and predictable, while Neighborhood C jumps up and down the most sharply from month to month. Neighborhood C has the most erratic consumption pattern, so it should be the priority target for a conservation campaign since usage there is hardest to predict.

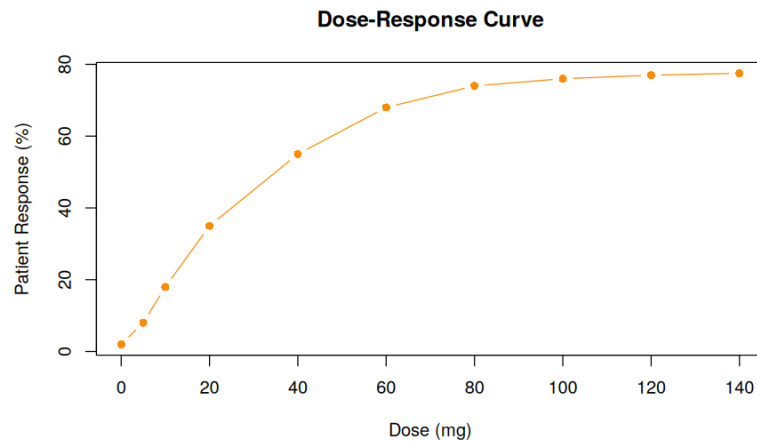
QUESTION 5: Dose–Response Curve – New Drug Trial (R)

1. Code:

```
dose <- c(0,5,10,20,40,60,80,100,120,140)
response <- c(2,8,18,35,55,68,74,76,77,77.5)
```

```
plot(dose, response, type = "b", pch = 16, col = "darkorange",
     xlab = "Dose (mg)", ylab = "Patient Response (%)",
     main = "Dose-Response Curve")
```

Output:



Interpretation:

Response rises steeply at first, then the curve flattens out from around 80mg onward, with almost no extra improvement between 100 and 140mg. This means the minimum effective dose is around 60–80mg, since increasing the dose beyond that gives little additional benefit but adds more risk.

QUESTION 6: Fertilizer Dose vs Crop Yield (Python)

1. Code:

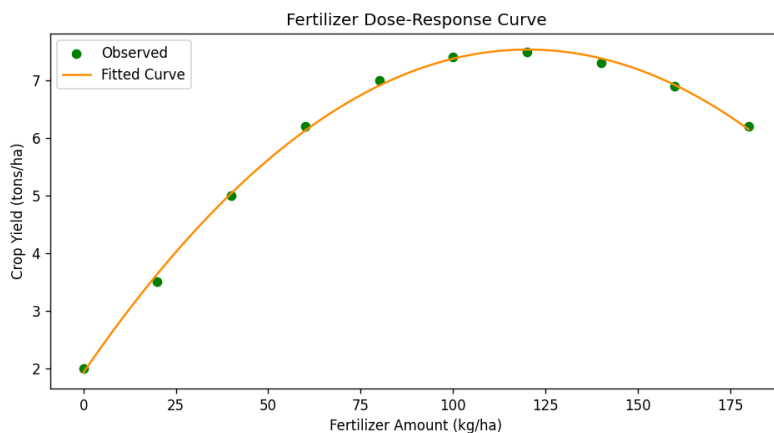
```
import numpy as np
import matplotlib.pyplot as plt

fert = np.array([0,20,40,60,80,100,120,140,160,180])
yield_ = np.array([2.0,3.5,5.0,6.2,7.0,7.4,7.5,7.3,6.9,6.2])

coeffs = np.polyfit(fert, yield_, 2)
curve = np.poly1d(coeffs)
x_smooth = np.linspace(0, 180, 200)

plt.scatter(fert, yield_, color="green", label="Observed")
plt.plot(x_smooth, curve(x_smooth), color="darkorange", label="Fitted Curve")
plt.xlabel("Fertilizer Amount (kg/ha)")
plt.ylabel("Crop Yield (tons/ha)")
plt.title("Fertilizer Dose-Response Curve")
plt.legend()
plt.show()
```

Output:



Interpretation:

Yield increases with fertilizer up to about 100–120 kg/ha, then the fitted curve peaks and starts to decline beyond that, showing over-application actually hurts yield. The optimal dosage range is about 100–120 kg/ha, which gives close to the maximum yield without wasting fertilizer.

QUESTION 7: STL Decomposition – Monthly Ticket Sales (R)

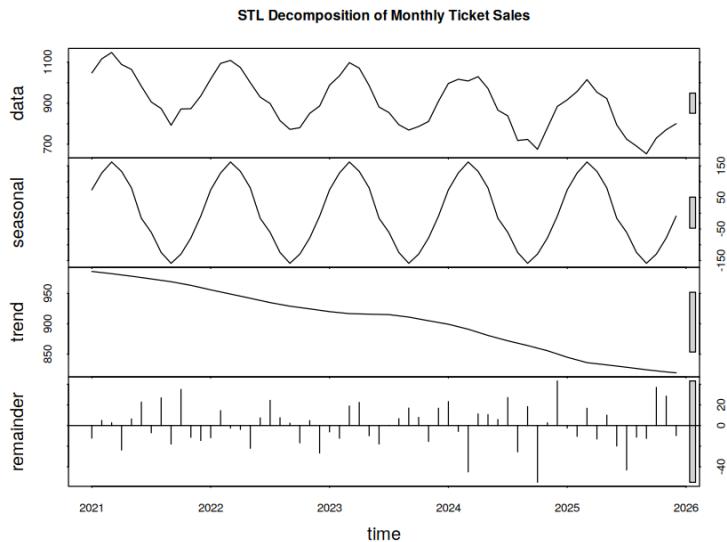
1. Code:

```
trend <- 1000 - (1:60)*3
seasonal <- 150*sin(2*pi*(1:60)/12)
sales <- trend + seasonal + rnorm(60, 0, 25)

sales_ts <- ts(sales, frequency = 12, start = c(2021,1))
decomp <- stl(sales_ts, s.window = "periodic")

plot(decomp, main = "STL Decomposition of Monthly Ticket Sales")
```

Output:



Interpretation:

The seasonal panel confirms a strong, repeating holiday-driven pattern each year, but once that is removed, the trend panel reveals a clear steady decline underneath. So the seasonality was indeed masking a real, slow underlying drop in ticket sales that the revenue team should investigate.

QUESTION 8: STL Decomposition – Weekly Active Users (Python)

1. Code:

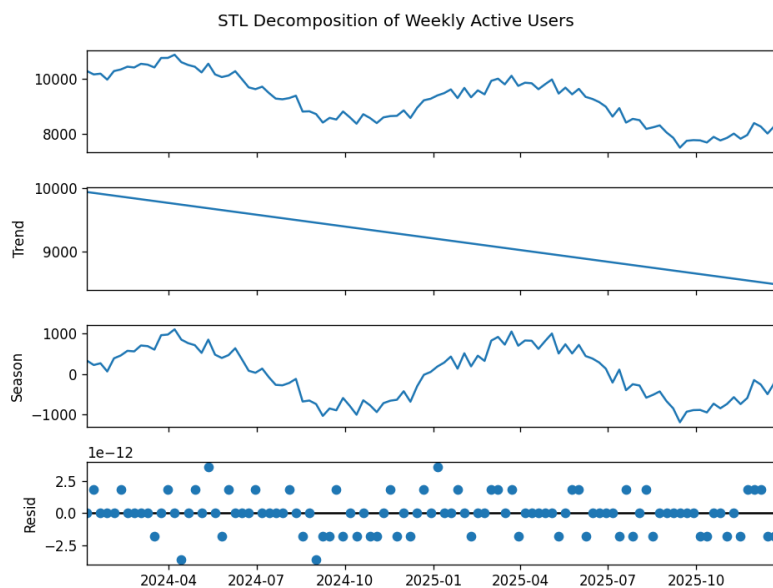
```
import pandas as pd
from statsmodels.tsa.seasonal import STL

trend = 10000 - np.arange(104)*15
seasonal = 800*np.sin(2*np.pi*np.arange(104)/52)
users = trend + seasonal + np.random.normal(0, 150, 104)

series = pd.Series(users, index=pd.date_range("2024-01-01", periods=104, freq="W"))
stl = STL(series, period=52)
result = stl.fit()
```

```
result.plot()
plt.show()
```

Output:



Interpretation:

The seasonal component shows users naturally dip and rise with a yearly cycle, which explains part of the recent drop. But the trend line is also declining steadily on its own, so the dip is not purely seasonal – there is a genuine underlying decline the product team should address.

QUESTION 9: 7-Day and 30-Day Moving Averages – Stock Price (R)

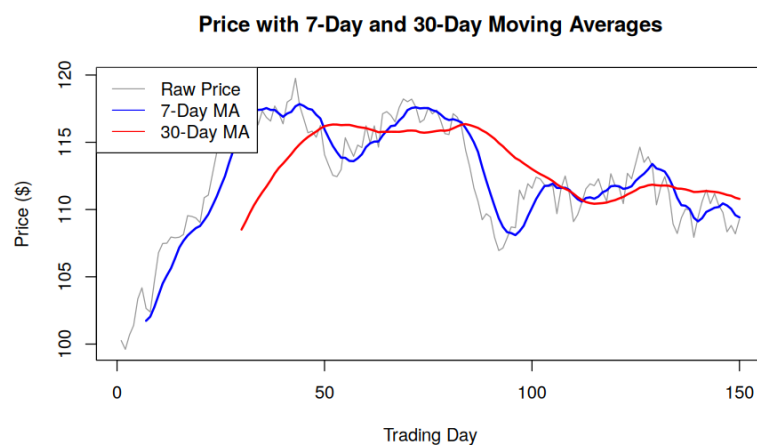
1. Code:

```
price <- 100 + cumsum(rnorm(150, 0, 1.2))
ma7 <- filter(price, rep(1/7, 7), sides = 1)
ma30 <- filter(price, rep(1/30, 30), sides = 1)

plot(price, type = "l", col = "grey60",
      xlab = "Trading Day", ylab = "Price ($)",
      main = "Price with 7-Day and 30-Day Moving Averages")
lines(ma7, col = "blue", lwd = 2)
lines(ma30, col = "red", lwd = 2)

legend("topleft", legend = c("Raw Price", "7-Day MA", "30-Day MA"),
      col = c("grey60", "blue", "red"), lty = 1)
```

Output:



Interpretation:

The 7-day average tracks the raw price closely and reacts fast, while the 30-day average is smoother and lags behind. When the 7-day line crosses above the 30-day line it suggests short-term momentum is turning upward (a buy signal), and a cross below suggests momentum is turning downward (a sell signal).

QUESTION 10: Detrending Quarterly GDP (Python)

1. Code:

```
import numpy as np
import matplotlib.pyplot as plt

quarters = np.arange(1, 41)
gdp = 500 + quarters*4 + 10*np.sin(quarters/2) + np.random.normal(0, 3, 40)

detrended = np.diff(gdp)

plt.plot(quarters[1:], detrended, color="purple", marker="o")
plt.axhline(0, color="grey", linestyle="--")
plt.xlabel("Quarter")
plt.ylabel("Detrended GDP (Quarter-over-Quarter Change)")
plt.title("Detrended GDP Series (First Differencing)")
plt.show()
```

Output:



Interpretation:

After removing the long-term growth trend by differencing, the remaining series bounces around zero instead of climbing steadily. These leftover ups and downs represent the business-cycle fluctuations – short expansions and contractions – that were previously hidden underneath the overall growth trend.